Trabalho de Inteligência Artificial — Problema dos Jarros d'Água

Aluno: Marcos Renan Costa Paiva

Curso: Tecnólogo em Análise e Desenvolvimento de Sistemas

Módulo: III

Disciplina: Inteligência Artificial

Professor: Rogério Figueredo de Sousa

1. Introdução

Neste trabalho, o problema dos jarros foi resolvido usando três algoritmos de busca: busca em largura (BFS), busca em profundidade (DFS) e busca com aprofundamento iterativo (IDDFS).

O problema usa dois jarros, um com capacidade para 4 litros e outro para 3 litros. Os dois começam vazios, e o objetivo é conseguir exatamente 2 litros em um deles. Para isso, o programa testa as ações permitidas e procura um caminho que chegue ao resultado.

2. Como o problema foi representado

Cada estado é representado por dois números: `(x, y)`. O primeiro número indica quantos litros estão no jarro de 4 litros, e o segundo indica a quantidade no jarro de 3 litros.

Por exemplo:

• `(0, 0)`: os dois jarros estão vazios;

• `(4, 0)`: o jarro de 4 litros está cheio e o de 3 litros está vazio;

• `(4, 2)`: o jarro de 4 litros está cheio e o outro contém 2 litros.

O estado inicial usado como padrão é `(0, 0)`. O programa também permite informar outro estado inicial, desde que as quantidades respeitem a capacidade de cada jarro.

O objetivo é atingido quando o primeiro ou o segundo jarro contém exatamente 2 litros. Em outras palavras, o teste é `x == 2 ou y == 2`.

As ações consideradas pelo programa são:

1. Encher o jarro de 4 litros.

2. Encher o jarro de 3 litros.

3. Esvaziar o jarro de 4 litros.

4. Esvaziar o jarro de 3 litros.

5. Passar água do jarro de 4 litros para o de 3 litros.

6. Passar água do jarro de 3 litros para o de 4 litros.

Na transferência, a água é passada até o jarro de origem ficar vazio ou o jarro de destino ficar cheio. O programa não considera ações que deixam o estado exatamente igual ao anterior.

Como o jarro de 4 litros pode conter de 0 a 4 litros e o de 3 litros pode conter de 0 a 3 litros, existem no máximo 20 estados diferentes.

3. Algoritmos usados

3.1 Busca em largura (BFS)

A busca em largura verifica primeiro os estados mais próximos do início e depois passa para os níveis seguintes. Ela usa uma fila para organizar os estados que ainda precisam ser analisados.

Como cada ação tem o mesmo custo, a BFS encontra uma solução com o menor número de ações. A desvantagem é que pode guardar muitos estados na memória.

3.2 Busca em profundidade (DFS)

A busca em profundidade segue um caminho antes de tentar outros. Ela usa uma pilha para controlar os estados que ainda serão explorados.

Esse método pode encontrar uma solução rapidamente, dependendo da ordem das ações, mas não garante que o caminho encontrado tenha o menor número de ações.

3.3 Aprofundamento iterativo (IDDFS)

O aprofundamento iterativo faz buscas em profundidade com limites cada vez maiores. Primeiro procura uma solução com limite zero, depois com limite um, depois com limite dois, e assim por diante.

Esse método pode encontrar uma solução de menor profundidade, como a BFS, mas repete parte do trabalho a cada novo limite. Por isso, pode expandir os mesmos estados em mais de uma rodada.

4. Funcionamento do programa

O código está no arquivo `main.py`. Ao executar, o programa recebe o estado inicial, roda os três algoritmos e mostra o caminho encontrado por cada um.

Também apresenta uma representação em texto da árvore de busca. Os estados que fazem parte do caminho da solução são marcados com `*`. No final, aparece uma tabela com o número de ações, os nós expandidos, os nós gerados e o tempo de execução de cada algoritmo.

Para executar com os dois jarros vazios, use:

```
python main.py --quatro 0 --tres 0
```

Se executar apenas `python main.py`, o programa pergunta as quantidades iniciais no terminal.

5. Comparação dos resultados

O número de ações representa quantos movimentos foram necessários para chegar ao objetivo. Já os nós expandidos são os estados que o algoritmo analisou, e os nós gerados são os estados novos que ele encontrou durante a busca.

Na execução de teste feita com o estado inicial `(0, 0)`, os resultados foram:

Algoritmo	Ações da solução	Nós expandidos	Nós gerados	Tempo observado (ms)
BFS	4	8	10	0,0235
DFS	6	7	9	0,0152
IDDFS	4	25	49	0,0783

Esses tempos são de uma execução específica e podem mudar de acordo com o computador e com as condições em que o programa é executado.

A BFS chegou à solução em quatro ações. O IDDFS também encontrou uma solução com quatro ações, mas precisou repetir buscas com limites diferentes, gerando e expandindo mais nós. A DFS encontrou uma solução válida com seis ações. Isso mostra que a DFS não garante o caminho mais curto, mesmo quando consegue chegar ao objetivo.

O caminho de quatro ações encontrado pela BFS e pelo IDDFS foi:

`(0, 0)` → `(0, 3)` → `(3, 0)` → `(3, 3)` → `(4, 2)`

Nesse caminho, primeiro o jarro de 3 litros é enchido. Depois, a água é transferida para o jarro de 4 litros. Em seguida, o jarro de 3 litros é enchido novamente e o conteúdo é transferido para o jarro de 4 litros até ele completar sua capacidade. Sobram exatamente 2 litros no jarro de 3 litros.

6. Conclusão

Com a implementação, foi possível resolver o problema dos jarros usando três estratégias de busca diferentes. A BFS encontrou uma solução com poucas ações e garante o menor caminho para esse tipo de problema, em que todas as ações têm o mesmo custo. A DFS encontrou uma solução válida, mas com mais ações. Já o IDDFS encontrou uma solução curta, porém repetiu parte da busca e gerou mais nós.

Assim, cada algoritmo tem suas características, e a comparação dos resultados ajuda a entender as diferenças entre eles.

7. Referência

Enunciado do trabalho de Inteligência Artificial, “Resolução do Problema dos Jarros D’Água por meio de Algoritmos de Busca sem Informação”, fornecido pelo professor Rogério Figueredo de Sousa.